UNIVERSITAT ROVIRA i VIRGILI



SDD 1 – Steering Files

TAP

Plan for the Two Labs

- **SDD 1 – Steering files** (today)
  - Vibe coding vs Spec Driven Development
  - What steering files and specs are
  - How to write and evaluate a good steering file
- **SDD 2 – Specs**
  - Writing specs for different types of task
  - Reviewing AI-generated specs and the cost in tokens

The Problem: AI Without Structure

- **Vibe coding**: asking an agent for code with one sentence and accepting whatever comes out
  - No context about the project
  - No criteria to decide whether the result is right
- Typical consequences
  - **Technical debt**: it works today, but ignores the project's conventions
  - **Hallucinations**: invented classes, methods or libraries
  - **Regressions**: it fixes one thing and breaks another
- **A fast agent without context produces fast mistakes**

The Core Idea of SDD

- **First we specify, then we implement**
- The quality of what we write determines the quality of what the agent generates
- The work shifts: less typing code, more **describing precisely** what we want and in which context
- An agent follows clear instructions well, but it is a poor guesser of vague ones

What Is a Steering File?

- **Stable context** about the project that the agent always receives
  - It changes rarely from one task to another
- Written by a person, **to be used by a machine**
  - The agent cannot ask questions: whatever is not written, it guesses
- Example (illustrative):

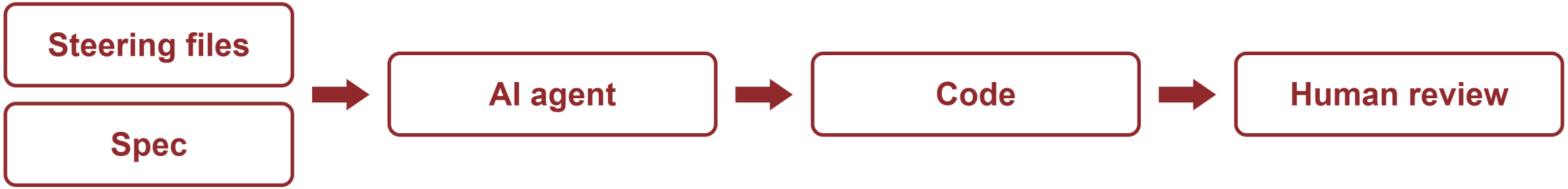
```
# Bookstore – project context
Stack: Java 17, Spring Boot
Layers: controller → service → repository
Rule: controllers never access repositories
Never: business logic in HTML templates
```

What Is a Spec?

- The description of **one concrete task** to be carried out
  - What we want, how we will check it, and what is left out
- Unlike a steering file, it is **per task**
  - It stops being useful once the task is done
- Example (illustrative):

```
# Search books by author
Goal: GET /books?author=name
Acceptance: returns only that author's books
Edge case: unknown author → empty list
Out of scope: pagination
```

Steering and Spec Together



- **Steering**: permanent context about the project
- **Spec**: one task, with its acceptance criteria
- The agent executes, but **a person checks** the result against the spec

Anatomy of a Steering File

- **Project context**: what the application does and for whom
- **Architecture**: layers and who talks to whom
- **Technologies**: language, framework and exact versions
- **Code conventions**: naming, packages, error handling
- **Constraints**: what must never be done, and why
- **Practical rule:** write what the agent cannot deduce by reading the code

Steering Quality Criteria

- **Coverage**: describes architecture, technologies and conventions
  - Is anything left for the agent to guess?
- **Specificity**: concrete rules and well-defined triggers
  - Would it be valid for any project? Then it is useless
- **Consistency**: no contradictions between steering files
  - Do two files ask for incompatible things?
- **Real usefulness**: an agent can use it without ambiguity
  - Could an agent act on this text alone?

Convention vs Defect

- The agent copies what it sees. The steering decides **what is worth copying**
- **Convention**: done on purpose; keep it and document it
  - Example: every controller returns a ResponseEntity
- **Defect**: it happens, but we do not want to repeat it
  - Example: a 300-line method inside a service
- **Inconsistent code**: the steering must choose the rule
  - Some services validate their input and others do not
- **If a defect is documented as a rule, the agent will spread it**

Triggers: When Is It Applied?

- **Always active**
  - Global context any task needs: project.md
- **By folder**
  - Rules for one area only: src/test/** → tests.md
- **By file type**
  - One guide per technology: *.java → java-backend.md
- **Common mistakes**
  - Too broad: loaded for every file, so more noise and more tokens
  - Overlapping: two steering files apply to the same file and may contradict

One Steering per File Type

- **Common context** (always active)
  - What the application is, its layers and its stack
- **Java backend**
  - Responsibilities of each layer, naming and packages, error handling
- **HTML templates**
  - Structure and names, how data arrives, no business logic in the view
- **Tests**
  - Test names and structure, what is mocked, how to run them
- **Consistency:** no steering contradicts the common context

Exercise: Find the Flaws

- A steering file, as written by a rushed team:

  trigger: always
  Write clean code and follow good practices.
  ---
  trigger: **/*.java
  Services access the database directly.
  ---
  trigger: src/**
  Services never access the database: they use repositories.

- Diagnose it with the four criteria:
  - Coverage, specificity, consistency, real usefulness
  - Write down each flaw and how you would fix it

Debrief: Flaws and Fixes

Flaw	Criterion	Fix
"Write clean code" says nothing concrete	Specificity	Verifiable rules: names, layers, errors
Two opposite rules about data access	Consistency	Keep one rule: services use repositories
Triggers **/*.java and src/** overlap	Specificity	One trigger per file, no shared zones
No architecture or stack described	Coverage, usefulness	Add context, layers and stack

- Almost every fix is the same: **turn a vague opinion into a verifiable rule**

UNIVERSITAT
ROVIRA i VIRGILI

